Generative AI Comprehensive Curriculum

Module 4: Language Models and Transformer-Based Generative Models

Ultra-Detailed Guide to Modern LLM Architecture

PART 1: LANGUAGE MODELS

What are Language Models?

Language Model (LM) = A probability distribution over sequences of words

Mathematical Definition:

$$P(\text{word}_1, \text{word}_2, \ldots, \text{word}_n)$$

The probability of a specific word sequence occurring in natural language.

How Language Models Work

Prediction Task: Given previous words, predict next word

```
Input: "The quick brown"
Model predicts probabilities:
  P("fox" | "The quick brown") = 0.45
  P("dog" | "The quick brown") = 0.25
  P("horse" | "The quick brown") = 0.15
  P("car" | "The quick brown") = 0.08
  ...

Select highest probability: "fox"
```

Recursive Generation:

```
Start: &lt;START&gt;
Output: "The"
Input: "&lt;START&gt; The"
Output: "quick"
Input: "&lt;START&gt; The quick"
Output: "brown"
Input: "&lt;START&gt; The quick brown"
Output: "fox"
...
Stop: &lt;END&gt;
```

Types of Language Models

1. Unidirectional Models (Left-to-Right)

Mechanism: Predict next token based on previous tokens only

Architecture:

```
Input:  "The cat sat"
Hidden:  ↓    ↓   ↓
         h1 → h2 → h3
                   ↓
Output:              "on"

Each hidden state uses only previous tokens (causal masking)
```

Mathematical Formulation:

$$P(x_t|x_1, \ldots, x_{t-1})$$

Examples:

- GPT series (GPT-2, GPT-3, GPT-4)
- LLaMA
- Falcon
- Claude

Advantages:

- Natural for text generation
- Efficient inference (generate left-to-right)
- Causal attention is interpretable

Disadvantages:

- Doesn't see future context
- Can't optimize bidirectionally

2. Bidirectional Models

Mechanism: Learn representations using both left and right context

Architecture:

```
Input:   "The cat sat on the"
         ↙  ↙  ↙  ↙  ↙
Context fusion in both directions
         ↖  ↖  ↖  ↖  ↖
Output: Rich contextualized embeddings
```

Mathematical Formulation:

$$\text{Representation}(x_t) = f(x_1, \ldots, x_{t-1}, x_t, x_{t+1}, \ldots, x_n)$$

Examples:

- BERT (Bidirectional Encoder Representations)
- RoBERTa
- ALBERT

Use Cases:

- Text classification
- Named entity recognition
- Sentiment analysis
- Question answering

Advantages:

- Better contextual understanding
- Excellent for classification tasks
- Pretrain on masked language modeling

Disadvantages:

- Can't generate text directly
- Requires task-specific fine-tuning
- Inference slower than autoregressive

3. Encoder-Decoder Models

Mechanism: Encode input, then decode output

Architecture:

```
English Input: "Hello world"
       ↓ Encoder (Understand)
    Context Vector (Compressed representation)
       ↓ Decoder (Generate)
French Output: "Bonjour le monde"
```

Mathematical Formulation:

$$P(y|x) = P(y_1|x) \times P(y_2|y_1, x) \times \ldots \times P(y_m|y_1 \ldots y_{m-1}, x)$$

Examples:

- T5 (Text-to-Text Transfer Transformer)
- BART

- Machine translation models
- Summarization models

Use Cases:

- Machine translation
- Summarization
- Abstractive question answering
- Paraphrase generation

The Transformer Architecture

Revolutionary Insight: "Attention is All You Need"

Problem with RNNs/LSTMs:

- Sequential processing (can't parallelize)
- Long-range dependencies suffer (vanishing gradients)
- Slow training on large datasets

Solution (2017): Transformer architecture based entirely on attention

Key Components

1. Self-Attention Mechanism

Idea: Each token attends to all other tokens in the sequence

Mathematical Formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Where:

- Q (Query): "What am I looking for?"
- K (Key): "What do I represent?"
- V (Value): "What information do I carry?"

Step-by-Step Process:

Step 1: Compute Query, Key, Value vectors

```
Input embedding of "cat": [0.1, 0.5, 0.3, 0.2]
Q = Input × W_q = [0.2, 0.3]  (What does "cat" query for?)
K = Input × W_k = [0.4, 0.1]  (What is "cat"?)
V = Input × W_v = [0.6, 0.2]  (What can "cat" contribute?)
```

Step 2: Compute attention scores

```
For each word in sequence:
  Score("cat", "the") = Q_cat · K_the = 0.2*0.4 + 0.3*0.1 = 0.11
  Score("cat", "cat") = Q_cat · K_cat = 0.2*0.4 + 0.3*0.1 = 0.11
  Score("cat", "sat") = Q_cat · K_sat = 0.2*0.4 + 0.3*0.1 = 0.11

Scores: [0.11, 0.11, 0.11]  (all equal initially)
```

Step 3: Apply softmax (convert to probabilities)

```
exp([0.11, 0.11, 0.11]) / sum = [0.33, 0.33, 0.33]

Without training: Equal attention to all words
With training: Learn which words to attend
```

Step 4: Weight and sum value vectors

```
Output = 0.33 * V_the + 0.33 * V_cat + 0.33 * V_sat
       = New representation of "cat"
         considering all context
```

Parallel Processing Advantage:

```
RNN/LSTM:
  Step 1: Process "the"     ← Must wait
  Step 2: Process "cat"     ← Must wait for step 1
  Step 3: Process "sat"     ← Must wait for step 2
  Total: 3 sequential steps

Attention:
  Step 1: Process all words in PARALLEL
  Step 2: Compute attention
  Total: 1 step (can run all in parallel on GPU)

Speedup: 3x for 3 words, 100x for 100 words!
```

2. Multi-Head Attention

Problem: Single attention might focus on single pattern

Solution: Multiple attention heads, each learning different patterns

```
Head 1: Focus on syntax relationships
Head 2: Focus on semantic relationships
Head 3: Focus on entity relationships
...
Head 8: Focus on other patterns

Results concatenated:
```

```
  [h1 output, h2 output, h3 output, ..., h8 output]

Benefit: Rich, multi-faceted representations
```

Mathematical Representation:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

Where each head:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

3. Feed-Forward Networks

Component: Simple neural network applied to each position

```
Hidden representation (from attention)
  ↓
Linear layer: d_model → d_ff (typically 4 × d_model)
  ↓
ReLU activation
  ↓
Linear layer: d_ff → d_model
  ↓
Output: Same shape as input
```

Formula:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Purpose:

- Non-linearity injection
- Position-wise processing
- Capacity increase

4. Positional Encoding

Problem: Attention is permutation-invariant (order-blind)

```
"The cat sat" and "sat cat The"
look the same to attention mechanism!
```

Solution: Add position information to embeddings

Formula:

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

$$PE(pos, 2i+1) = \cos(pos/10000^{2i/d_{model}})$$

Result: Each position has unique encoding, preserving order

Complete Transformer Block

```
Input Embedding
  ↓
Add Positional Encoding
  ↓
┌──────────────────────────────────┐
│ Transformer Layer (repeat n) │
├──────────────────────────────────┤
│ Multi-Head Attention         │
│ + Layer Normalization + Add  │
│ ↓                            │
│ Feed-Forward Network         │
│ + Layer Normalization + Add  │
└──────────────────────────────────┘
  ↓
Output (contextualized embeddings)
```

GPT Architecture (Decoder-Only)

Stack of Transformer Blocks:

```
Input: "The quick brown"
  ↓
Embedding + Positional Encoding
  ↓
[Block 1] Self-Attention + FFN
  ↓
[Block 2] Self-Attention + FFN
  ↓
... (12-96 blocks depending on model size)
  ↓
[Block n] Self-Attention + FFN
  ↓
Output Projection to Vocabulary
  ↓
Softmax Distribution over 50,000 words
  ↓
Prediction: "fox" (highest probability)
```

BERT Architecture (Encoder-Only)

For Understanding (Classification):

```
Input: "The cat sat"
  ↓
Embedding + Positional Encoding + Segment Encoding
  ↓
[Block 1] Bidirectional Self-Attention + FFN
```

```
  ↓
[Block 2] Bidirectional Self-Attention + FFN
  ↓
... (12-24 blocks)
  ↓
[CLS] Token Embedding (aggregate)
  ↓
Output: Classification or Sequence Label
```

Advanced Transformer Models

GPT Evolution

GPT-1 (2018):

- 117M parameters
- 1,000 training steps per iteration
- Task: Next token prediction

GPT-2 (2019):

- 1.5B parameters
- Could do multiple tasks (few-shot learning)
- Raised safety concerns (too capable)

GPT-3 (2020):

- 175B parameters
- In-context learning (few-shot examples in prompt)
- Emergence of abilities (reasoning, programming)

GPT-4 (2023):

- 1.76T parameters (estimated)
- Multimodal (text + images)
- Better reasoning and reduced hallucinations
- MMLU score: 86.5% (vs 93% human)

BERT Evolution

BERT (2018):

- 110M parameters
- Bidirectional training
- Masked language modeling

RoBERTa (2019):

- Better pretraining
- Better evaluation methodology
- Improved GLUE score from 72.1 → 88.5

ALBERT (2020):

- 70M parameters (16x smaller than BERT)
- 18x faster training
- Same performance
- Demonstrates efficiency improvements

Other Notable Models

T5: Text-to-Text framework

- Frames all NLP tasks as text-to-text
- Single model for translation, summarization, QA, etc.

LLaMA: Open-source alternative to GPT

- 7B, 13B, 33B, 65B parameters
- Trained on different data
- Good alternative to proprietary models

Claude: Anthropic's model

- Constitutional AI training
- Better factuality and reduced hallucinations
- Safer by design

Applications of Transformer Models

1. Machine Translation

End-to-End Learning:

```
Input English: "Hello world"
  ↓ Encoder Transformer
  Understand meaning
  ↓ Decoder Transformer
  Generate French: "Bonjour le monde"

Pre-transformer: 30-40% BLEU score
With Transformer: 45-50% BLEU score
Improvement: 25-30%
```

2. Text Classification

Example - Sentiment Analysis:

```
BERT for Sentiment:
  Input: "This movie was amazing!"
  ↓ BERT Encoder
  ↓ Classification Head
  Output: Positive (0.95 confidence)

Applications:
  - Review analysis
  - Customer feedback
  - Social media monitoring
```

3. Question Answering

Reading Comprehension:

```
Context: "Paris is the capital of France.
          It is known for the Eiffel Tower."

Question: "What is the capital of France?"

Model:
  1. Reads context with transformer
  2. Understands question with transformer
  3. Locates answer span
  Output: "Paris"
```

4. Summarization

Abstractive Summarization:

```
Input Document (1000 words):
  "Tesla announced quarterly earnings.
   Revenue up 30%. Profit margins improved.
   Elon Musk expressed optimism on AI..."

T5 Model:
  Summarize this text

Output Summary (50 words):
  "Tesla reported 30% revenue growth with
   improved margins. CEO optimistic about AI prospects."

User Value: Read summary in 30 seconds vs 10 minutes
```

Key Innovations in Transformers

1. Scaled Dot-Product Attention

Insight: Scale by square root of dimension

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Why Scaling?: Prevents attention scores from becoming too large/small

2. Layer Normalization

Purpose: Stabilize training in deep networks

```
Before: Hidden values range: [-100, 100]
After: Hidden values range: [-1, 1]
Result: Stable gradients, faster convergence
```

3. Residual Connections

Innovation: Skip connections preserve information flow

```
Output = Attention(Input) + Input
         ↑ Allows gradient to flow directly
         ↑ Prevents information bottleneck

Benefit: Can train very deep models (100+ layers)
```

4. Causal Attention (For Generation)

Mechanism: Mask future positions during generation

```
Position 1: Can only attend to position 1
Position 2: Can only attend to positions 1-2
Position 3: Can only attend to positions 1-3

Result: Can't "cheat" by looking at future
```

Mathematical Deep Dive: Attention Mechanism

The Problem Attention Solves

RNN Limitation:

```
"The quick brown fox jumped over the lazy dog"
                                  ↑
RNN at this position must remember "fox" from 6 positions back

Gradient must flow through 6 steps:
∂Loss/∂input = ∂Loss/∂out · ∂out/∂h6 · ∂h6/∂h5 · ... · ∂h1/∂input

Each multiplication shrinks gradient (vanishing gradient problem)
```

Attention's Solution

```
"The quick brown fox jumped over the lazy dog"
                                  ↑
Attention at "dog" position:
Direct connection to all previous words
No sequential path → No vanishing gradients
```

Attention Computation Detailed

Given:

- Query matrix $Q \in \mathbb{R}^{n \times d_k}$
- Key matrix $K \in \mathbb{R}^{n \times d_k}$
- Value matrix $V \in \mathbb{R}^{n \times d_v}$

Compute:

1. **Similarity scores**:

$$S = QK^T \in \mathbb{R}^{n \times n}$$

Each entry $S_{ij} = \text{Similarity}(\text{Query}_i, \text{Key}_j)$

2. **Scaling** (for stability):

$$S_{scaled} = \frac{S}{\sqrt{d_k}}$$

3. **Softmax** (convert to probabilities):

$$A = \text{softmax}(S_{scaled}) \in \mathbb{R}^{n \times n}$$

Each row sums to 1: $\sum_j A_{ij} = 1$

4. **Weighted sum**:

$$\text{Output} = AV \in \mathbb{R}^{n \times d_v}$$

Each output position is weighted sum of all values

Attention Visualizations

Example: "It makes sense" Query

```
Query Word: "It"

Attention to each word:
"It"      : 0.45  ═══════════════════
"makes"   : 0.25  ═══════════
"sense"   : 0.30  ═══════════════

"It" attends mostly to itself (understanding self)
Then to "sense" (understanding relationship)
Less to "makes" (less relevant)
```

Example: "The bank manager" Disambiguation

```
Token: "bank"

Scenario 1 (Financial):
"The bank manager"
  - "bank" attention to "manager": 0.7
  - Interprets as: Financial institution

Scenario 2 (Geography):
"Along the river bank manager"
  - "bank" attention to "river": 0.7
  - Interprets as: River bank

Same word, different attention → Different meaning!
```

Training Transformer Models

Pre-training Objectives

Autoregressive (GPT):

```
Minimize: -Σ log P(word_i | word_1, ..., word_{i-1})

Learning: Predict each word from previous words
```

Masked Language Modeling (BERT):

```
Mask 15% of words randomly
Minimize: -Σ log P(masked_word | context)
```

```
Learning: Predict masked words from bidirectional context
```

Fine-tuning

General Pre-train → Specific Task:

```
Pre-trained: Language model (generic knowledge)
↓
Fine-tune on:
  - Sentiment classification (task-specific)
  - Named entity recognition
  - Question answering
  - etc.

Benefit: Transfer learning (few examples needed)
```

References

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2023). Attention is all you need. In Advances in neural information processing systems (pp. 5998-6008).
2. Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2023). Language models are few-shot learners. In Advances in neural information processing systems (pp. 1877-1901).
3. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2023). BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers) (pp. 4171-4186).
4. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2023). Language models are unsupervised multitask learners. OpenAI blog, 1(8), 9.
5. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., ... & Liu, P. J. (2023). Exploring the limits of transfer learning with a unified text-to-text transformer. In International Conference on Machine Learning (pp. 8023-8035). PMLR.
6. Openai. (2023). GPT-4 Technical Report. arXiv preprint arXiv:2303.08774.
7. Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., ... & Kerkez, B. (2023). Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288.
8. Anthropic. (2023). Constitutional AI: Harmlessness from AI Feedback. arXiv preprint arXiv:2212.08073.
9. Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, Z., Chen, S., ... & Schwenk, H. (2023). OPT: Open Pre-trained Transformer Language Models. arXiv preprint arXiv:2205.01068.
10. Wei, J., Bosma, M., Zhao, Y., Gur-Arie, K., Yuhuai, W., Jie, T., ... & Le, Q. V. (2023). Emergent abilities of large language models. Transactions on Machine Learning Research, 1(2).

Next Steps

This module provides the foundation for understanding all modern language models. In subsequent modules, you'll learn:

Module 5: Prompt Engineering - How to effectively communicate with LLMs
Module 6: Fine-tuning - Customizing models for specific tasks
Module 8: LangChain - Building applications with LLMs
Module 9: RAG - Augmenting LLMs with external knowledge

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20]

⁂

1. https://ai.google/principles/
2. https://arxiv.org/abs/2312.10997
3. https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api
4. https://www.promptingguide.ai/techniques
5. https://www.k2view.com/blog/prompt-engineering-techniques/
6. https://aws.amazon.com/blogs/machine-learning/prompt-engineering-techniques-and-best-practices-learn-by-doing-with-anthropics-claude-3-on-amazon-bedrock/
7. https://arxiv.org/html/2507.03405v1
8. https://ubiai.tools/the-complete-guide-to-generative-adversarial-networks-gans/
9. https://arxiv.org/html/2411.14511v1
10. https://dextralabs.com/blog/prompt-engineering-for-llm/
11. https://www.artificialintelligencepub.com/jairi/article/view/jairi-aid1004/5887
12. https://dl.acm.org/doi/10.1145/3613904.3642466
13. https://www.sciencedirect.com/science/article/pii/S2666827024000483
14. https://www.ibm.com/think/topics/generative-ai
15. https://aiverifyfoundation.sg/wp-content/uploads/2024/05/Model-AI-Governance-Framework-for-Generative-AI-May-2024-1-1.pdf
16. https://www.centraleyes.com/generative-ai-governance/
17. https://www.geeksforgeeks.org/nlp/transformer-attention-mechanism-in-nlp/
18. https://www.promptingguide.ai/research/rag
19. https://www.informationpolicycentre.com/uploads/5/7/1/0/57104281/cipl_applying_data_protection_principles_genai_dec24.pdf
20. https://arxiv.org/abs/1706.03762